



15. Hash Tables

"Find the data without searching the entire collection."

Previous Section:

 [14. Sorting Algorithms](#)

Contents:

[1. What is a Hash Table?](#)

[2. Applications of Hash Tables](#)

[Advantages](#)

[Disadvantages](#)

[3. Operations on Hash Tables](#)

[Summary](#)

[References](#)

So far, we have studied several fundamental data structures such as Arrays, Linked Lists, Stacks, Queues, Trees, and Heaps. Each of these structures stores and organizes data in different ways, but they all have one common limitation: searching for a specific element can become increasingly expensive as the amount of data grows.

What if we could access a piece of data almost instantly, without scanning the entire collection?

This is exactly the problem that **Hash Tables** are designed to solve.

A Hash Table stores information as **key-value pairs** and uses a special mathematical function called a **Hash Function** to determine where each piece of data should be stored. Instead of searching through every element one by one, we simply calculate the position of the key and retrieve the value directly.

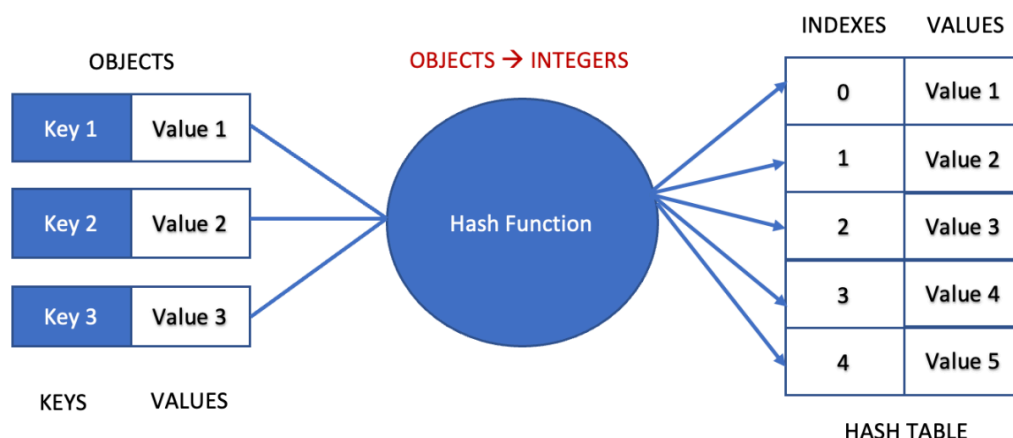
When designed properly, most operations such as insertion, searching, and deletion can be performed in **constant time $O(1)$** , making Hash Tables one of the most efficient and widely used data structures in computer science.

1. What is a Hash Table?

Before discussing Hash Tables, we first need to understand the meaning of the word **hash**. A **hash** is the result produced by a **Hash Function**, which converts a key into an integer value. This integer is then used as an index inside the table.

A **Hash Table** is a data structure that stores information as **key-value pairs**. Internally, it is simply an array where each position (called a **bucket**) stores one or more objects.

Each stored object consists of two parts:



- **Key** – used to identify the object
- **Value** – the actual information associated with the key

Instead of searching every element sequentially, the Hash Function computes an index where the object should be stored.

The overall process is called **Hashing**.



	v
+-----+-----+	
Index	Stored Object (Key, Value)
+-----+-----+	
0	("John", 82)
1	Empty
2	("Mary", 95)
3	Empty
4	("David", 71)
+-----+-----+	

Each stored object consists of a **Key** and a **Value**.

Stored Object		
+-----+		
Key	Value	
+-----+		
John	82	
+-----+		

For example,

Key = "John"	
	Hash Function
	v
h("John") = 4	
Store at Index 4	

Instead of searching every entry, we immediately know where the object should be stored.

2. Applications of Hash Tables

Hash Tables are among the most widely used data structures in computer science because of their fast lookup speed. Some common applications include:

- Databases: Databases use Hash Tables to quickly locate records using unique identifiers such as customer IDs or student IDs.
- Caches: Caches store recently accessed data so that future requests can be retrieved immediately.
- Symbol Tables: Compilers use Hash Tables to store variables, functions, and their corresponding information.
- Cryptography: Hash functions are widely used to verify file integrity, generate digital signatures, and securely store passwords.
- Distributed Systems: Load balancers use hashing algorithms to distribute requests among multiple servers.
- File Systems: Operating systems use Hash Tables to locate files and storage blocks efficiently.

Advantages

- Very fast lookup (typically $O(1)$)
- Fast insertion and deletion
- Supports many different data types
- Memory efficient compared to many tree structures
- Includes collision handling techniques

Disadvantages

- Performance decreases when many collisions occur
- Collisions cannot be completely avoided
- Does not preserve the order of elements
- Has a limited capacity before resizing becomes necessary
- More difficult to implement than arrays or linked lists

3. Operations on Hash Tables

Like most data structures, Hash Tables support several basic operations:
Create; Insert; Search; Delete; Update

Before learning how hashing improves performance, let's compare it with ordinary data structures.

- Case 1 — Unsorted Sequential Array

Suppose the following records are stored in an ordinary array.

```
Array
Index
  0      1      2      3
+-----+ +-----+ +-----+ +-----+
|14|1234| |45|2345| |19|3456| |27|4567|
+-----+ +-----+ +-----+ +-----+
```

To find **Key = 19**, we search one element at a time.

Search Key = 19

Index 0

```
+-----+
|14|1234|
+-----+
  x
```

↓

Index 1

```
+-----+
|45|2345|
+-----+
  x
```

↓

Index 2

```
+-----+
|19|3456|
+-----+
  ✓ Found
```

Since we may have to examine every element, searching requires **Time Complexity** = $O(n)$

- Time Complexity

Operation	Complexity
Initialize	$O(1)$
Is Empty	$O(1)$
Insert	$O(1)$
Find	$O(n)$
Remove	$O(n)$

- Case 2 — Sorted Sequential Array

Now suppose the array is sorted by key.

```

Array
Index
    0          1          2          3
+-----+ +-----+ +-----+ +-----+
|14|1234| |19|3456| |27|4567| |45|2345|
+-----+ +-----+ +-----+ +-----+
  
```

Since the keys are sorted, we can use **Binary Search**.

```

Search Key = 27
  
```

```

Step 1
  
```

```

Middle = 19
  
```

```

14  19  27  45
   ^
27 > 19
  
```

```

Search Right Half
  
```

```

-----
  
```

```

Step 2
  
```

```

27  45
 ^
  
```

Found

Searching now improves to $O(\log n)$ although insertion and deletion still require shifting elements.

- Time Complexity

Operation	Complexity
Initialize	$O(1)$
Is Empty	$O(1)$
Insert	$O(n)$ (elements may need to shift)
Find	$O(\log n)$
Remove	$O(n)$

Note: Although the original table lists insertion as $O(1)$, maintaining a sorted array requires shifting elements, so insertion is generally $O(n)$.

- Case 3 — Sorted Linked List

Suppose the same records are stored in a linked list.

Head

|
v

```
+-----+ +-----+ +-----+ +-----+
|14|1234| -> |19|3456| -> |27|4567| -> |45|2345| -> NULL
+-----+ +-----+ +-----+ +-----+
```

Searching still requires traversing nodes one by one.

Head

|
v

14 ---> 19 ---> 27 ----> 45

^
Found

Since linked lists do not support direct indexing, Binary Search cannot be efficiently used.

- Time Complexity

Operation	Complexity
Initialize	$O(n)$
Is Empty	$O(1)$
Insert	$O(n)$
Find	$O(n)$
Remove	$O(n)$

- Why Use a Hash Table?

Instead of searching sequentially, we compute the location immediately.

Search Key = 27

27
|
| Hash Function
v

$h(27) = 3$

|

Go directly to Index 3

Index

0
1
2
3 ---> (27, 4567) ✓
4
5
6

No sequential search is required. This is why Hash Tables achieve an average lookup time of $O(1)$.

Summary

Hash Tables are one of the fastest data structures for storing and retrieving information. Instead of searching through every element, they use a **Hash Function** to convert a key into an array index, allowing data to be accessed almost instantly. Because of this, insertion, searching, and deletion are typically performed in **constant time $O(1)$** , making Hash Tables an essential data structure in databases, caches, compilers, operating systems, and many modern software applications.

However, Hash Tables are not perfect. Their performance depends heavily on the quality of the Hash Function, and collisions can reduce their efficiency if not handled properly.

In the next section, we will explore the two concepts that make Hash Tables possible: **Hashing** and **Hash Functions**. We will learn how keys are converted into indices, why collisions occur, and how different hashing techniques minimize these collisions while maintaining fast performance.

References

https://drive.google.com/file/d/1le3JCsQzFeq4MydZEYR2XLKUISUaUypv/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/hash-table-data-structure/>

<https://www.programiz.com/dsa/hash-table>

<https://www.youtube.com/watch?v=FsfRsGFHuv4>

Next Section:

 [15.1. Hashing and Hash Functions](#)